

École Nationale des Sciences Appliquées d'Al Hoceima

Filière : TDIA – 2^{ème} Année

Rapport TP6 – JEE

Application Web avec Architecture
MVC2, Spring MVC et Hibernate

Réalisé par :

Yassir HAMRI

Encadré par :

**Pr. Mohamed
CHERRADI**

Année Universitaire : 2025/2026

Table des matières

1	Mise en contexte	3
2	Gestion des dépendances (pom.xml)	3
3	Mise en place des modèles et du DAO	4
3.1	Configuration de l'entité Produit	4
3.2	Configuration d'Hibernate	4
3.3	Implémentation du DAO	5
4	Couche Service (Métier)	5
5	Inversion de contrôle avec Spring	6
5.1	Le contexte applicatif	6
5.2	Configuration de la Servlet	6
5.3	Descripteur de déploiement	7
6	Développement de l'interface	7
6.1	Le Contrôleur principal	8
6.2	Fichier de présentation	8
7	Conclusion	9

1 Mise en contexte

Ce document présente l'implémentation d'une application Java Enterprise Edition (JEE) basée sur l'architecture Spring MVC. Le travail principal a consisté à faire évoluer un projet existant vers ce framework afin d'optimiser sa structure. J'ai conservé l'outil Hibernate pour assurer la gestion de la base de données. L'adoption de Spring permet de clarifier le code en remplaçant l'utilisation directe des servlets par un contrôleur frontal unifié. Les sections suivantes détaillent les différentes étapes techniques réalisées pour aboutir à ce résultat.

2 Gestion des dépendances (pom.xml)

La première étape de la réalisation a été la configuration du fichier Maven. Il a fallu importer les bibliothèques adéquates pour le fonctionnement conjoint de Spring Web et de l'outil de persistance. La sélection des versions 5.3.30 pour l'écosystème Spring et 3.5.0-Final pour Hibernate s'est avérée nécessaire pour garantir la stabilité de l'application et éviter des conflits internes.

```
1 <dependencies>
2   <dependency>
3     <groupId>org.hibernate</groupId>
4     <artifactId>hibernate-core</artifactId>
5     <version>3.5.0-Final</version>
6     <scope>compile</scope>
7   </dependency>
8   <dependency>
9     <groupId>org.springframework</groupId>
10    <artifactId>spring-webmvc</artifactId>
11    <version>5.3.30</version>
12  </dependency>
13  <dependency>
14    <groupId>org.springframework</groupId>
15    <artifactId>spring-context</artifactId>
16    <version>5.3.30</version>
17  </dependency>
18  <!-- JSTL et MySQL -->
19  <dependency>
20    <groupId>javax.servlet</groupId>
21    <artifactId>jstl</artifactId>
22    <version>1.2</version>
23  </dependency>
24  <dependency>
25    <groupId>mysql</groupId>
26    <artifactId>mysql-connector-java</artifactId>
27    <version>8.0.33</version>
28  </dependency>
29 </dependencies>
```

3 Mise en place des modèles et du DAO

Après la configuration du projet, j'ai poursuivi avec le traitement des données. Il a fallu lier la logique objet à la base de données relationnelle.

3.1 Configuration de l'entité Produit

La classe définissant le produit a été modifiée pour devenir une entité au sens de la norme JPA. L'ajout des annotations correspondantes permet de spécifier la table et de marquer l'attribut servant de clé primaire.

```
1 package DAO;
2 import javax.persistence.*;
3 import java.io.Serializable;
4
5 @Entity
6 public class Produit implements Serializable {
7     @Id
8     @GeneratedValue(strategy=GenerationType.IDENTITY)
9     private Long idProduit;
10    private String nom;
11    private String description;
12    private Double prix;
13
14    public Produit() {
15    }
16
17    public Produit(String nom, String description, Double prix) {
18        this.nom = nom;
19        this.description = description;
20        this.prix = prix;
21    }
22    // Getters et Setters...
23 }
```

3.2 Configuration d'Hibernate

Pour assurer la liaison avec le système de gestion de base de données MySQL local, j'ai complété le fichier de paramètres.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!DOCTYPE hibernate-configuration PUBLIC
3     "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4     "http://hibernate.sourceforge.net/hibernate-configuration-3.0.
5     dtd">
6 <hibernate-configuration>
7     <session-factory>
8         <property name="hibernate.connection.driver_class">com.mysql.cj.
9             jdbc.Driver</property>
10        <property name="hibernate.connection.url">jdbc:mysql://
11            localhost:3306/gestion_produits_tp6?createDatabaseIfNotExist=
12            true&amp;</property>
13        <property name="hibernate.connection.username">root</property>
14        <property name="hibernate.connection.password"></property>
15        <property name="hibernate.dialect">org.hibernate.dialect.
16            MySQL5Dialect</property>
```

```

12     <property name="hibernate.hbm2ddl.auto">update</property>
13     <mapping class="DAO.Produit"/>
14 </session-factory>
15 </hibernate-configuration>

```

La gestion des sessions s'effectue via une classe utilitaire que j'ai conservée telle quelle par rapport aux exercices précédents.

3.3 Implémentation du DAO

Les méthodes permettant de manipuler les données sont d'abord listées dans une interface. Leur code effectif se trouve dans la classe d'implémentation. Ce code utilise les fonctions transactionnelles standards. J'ai également intégré une procédure d'initialisation pour générer un contenu initial dans la base au moment de l'exécution.

```

1 package DAO;
2
3 import Utils.HibernateUtils;
4 import java.util.List;
5 import org.hibernate.*;
6
7 public class ProduitImpl implements ProduitDAO {
8
9     public void init() {
10         System.out.println("spring ioc worked");
11         addProduit(new Produit("PC 1", "Sony vaio 1", 7000.0));
12         addProduit(new Produit("PC 2", "Sony vaio 2", 6000.0));
13     }
14
15     @Override
16     public void addProduit(Produit p) {
17         Session session = HibernateUtils.getSessionFactory().openSession
18             ();
19         Transaction tr = null;
20         try {
21             tr = session.beginTransaction();
22             session.save(p);
23             tr.commit();
24         } catch (Exception e) {
25             if (tr != null) tr.rollback();
26             e.printStackTrace();
27         } finally {
28             session.close();
29         }
30     }
31     // Les methodes de mise a jour, de suppression et de recuperation
32     suivent ce meme modele transactionnel.
33 }

```

4 Couche Service (Métier)

Le maintien d'une bonne architecture applicative exige de ne pas accéder directement aux données depuis l'interface utilisateur. J'ai donc rédigé une couche intermédiaire

appelée "Service". Cette composante s'occupera plus tard de recevoir l'objet d'accès aux données grâce aux mécanismes d'injection de Spring.

```
1 package services;
2
3 import java.util.List;
4 import DAO.Produit;
5 import DAO.ProduitDAO;
6
7 public class ProduitImplMetier implements ProduitMetier {
8
9     private ProduitDAO dao;
10
11     // Cette methode est essentielle pour que le fichier XML puisse
12     // faire le lien.
13     public void setDao(ProduitDAO dao) {
14         this.dao = dao;
15     }
16
17     public void addProduit(Produit p) {
18         dao.addProduit(p);
19     }
20     // Le reste du code propage simplement l'information vers la methode
21     // DAO.
22 }
```

5 Inversion de contrôle avec Spring

La configuration de Spring MVC repose sur plusieurs fichiers XML qui permettent d'orchestrer les différents composants du programme.

5.1 Le contexte applicatif

Le fichier de définition des composants permet d'instancier les classes et d'organiser les liaisons entre elles. J'y ai déclaré la couche de données et la couche métier. L'instruction permet également d'exécuter la méthode d'insertion initiale automatiquement.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans" ...>
3     <bean id="objDAO" class="DAO.ProduitImpl" init-method="init"/>
4
5     <bean id="objServices" class="services.ProduitImplMetier">
6         <property name="dao" ref="objDAO"/>
7     </bean>
8 </beans>
```

5.2 Configuration de la Servlet

Dans le second fichier de configuration, j'ai spécifié à Spring de localiser seul les contrôleurs présents dans le code source. J'ai configuré l'objet chargé de résoudre les chemins pour simplifier la localisation des interfaces graphiques finales.

```

1 <beans xmlns="http://www.springframework.org/schema/beans"
2     xmlns:context="http://www.springframework.org/schema/context" ...
3     >
4     <context:component-scan base-package="controller"/>
5
6     <bean class="org.springframework.web.servlet.view.
7         InternalResourceViewResolver">
8         <property name="prefix" value="/Pages/" />
9         <property name="suffix" value=".jsp" />
10    </bean>
11 </beans>

```

5.3 Descripteur de déploiement

Le descripteur central a été mis à jour pour rediriger l'ensemble du trafic vers l'objet fourni par le framework. J'ai complété cette étape en activant l'écouteur permettant de charger les instances Java dès le démarrage du serveur.

```

1 <web-app xmlns="http://java.sun.com/xml/ns/javaee" ... version="3.0">
2     <context-param>
3         <param-name>contextConfigLocation</param-name>
4         <param-value>/WEB-INF/spring-beans.xml</param-value>
5     </context-param>
6     <listener>
7         <listener-class>org.springframework.web.context.
8             ContextLoaderListener</listener-class>
9     </listener>
10    <servlet>
11        <servlet-name>action</servlet-name>
12        <servlet-class>org.springframework.web.servlet.DispatcherServlet
13        </servlet-class>
14        <init-param>
15            <param-name>contextConfigLocation</param-name>
16            <param-value>/WEB-INF/application-servlet-config.xml</param-
17            value>
18        </init-param>
19        <load-on-startup>1</load-on-startup>
20    </servlet>
21    <servlet-mapping>
22        <servlet-name>action</servlet-name>
23        <url-pattern>*.aspx</url-pattern>
24    </servlet-mapping>
25 </web-app>

```

6 Développement de l'interface

La réalisation de l'interface utilisateur constitue la dernière phase du projet.

6.1 Le Contrôleur principal

J'ai structuré la classe chargée d'interpréter les requêtes. L'objet contenant la logique métier y est automatiquement injecté par l'annotation adéquate. Chaque fonctionnalité du site est ensuite traitée par une méthode dédiée.

```
1 package controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Controller;
5 import org.springframework.ui.Model;
6 import org.springframework.web.bind.annotation.RequestMapping;
7 import org.springframework.web.bind.annotation.RequestMethod;
8 import org.springframework.web.bind.annotation.RequestParam;
9 import DAO.Produit;
10 import services.ProduitMetier;
11
12 @Controller
13 public class ProduitController {
14
15     @Autowired
16     ProduitMetier services;
17
18     @RequestMapping(value="/index")
19     public String pageIndex(Model model) {
20         model.addAttribute("listeProduit", services.getAllProduits());
21         return "produits"; // Sera prefixe et suffixe par le
22             ViewResolver
23     }
24
25     @RequestMapping(value="/addProduct")
26     public String addProduct(Model model, Produit p) {
27         services.addProduit(p);
28         model.addAttribute("listeProduit", services.getAllProduits());
29         return "produits";
30     }
31
32     // Le meme principe est applique pour l'edition et la suppression
33     // des enregistrements.
34 }
```

6.2 Fichier de présentation

J'ai construit le fichier de présentation dans son dossier de sécurité. Ce fichier génère dynamiquement les tableaux d'affichage grâce à la bibliothèque standard. Le formulaire a été pensé pour s'adapter aussi bien lors d'une nouvelle saisie que lors d'une modification de produit existant.

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4 <!--DOCTYPE html-->
5 <html>
6 <head>
7   <title>Gestion des Produits</title>
8 </head>
```



```

8 <body>
9 <h1>Gestion des produits | Spring MVC & Hibernate</h1>
10
11 <form action="{produitEdit != null ? 'updateProduit' : 'addProduct
12   '}.aspx" method="post">
13   <input type="hidden" name="idProduit" value="{produitEdit !=
14     null ? produitEdit.idProduit : ''}" />
15   Nom : <input type="text" name="nom" value="{produitEdit != null
16     ? produitEdit.nom : ''}" />
17   Description : <input type="text" name="description" value="{
18     produitEdit != null ? produitEdit.description : ''}" />
19   Prix : <input type="text" name="prix" value="{produitEdit !=
20     null ? produitEdit.prix : ''}" />
21   <input type="submit" value="{produitEdit != null ? 'Mettre à
22     jour' : 'Ajouter'}" />
23 </form>
24
25 <table border="1" width="50%">
26   <tr>
27     <th>ID</th> <th>NOM</th> <th>DESCRIPTION</th> <th>PRIX</th>
28     <th>Option</th>
29   </tr>
30   <c:forEach items="{listeProduit}" var="o">
31     <tr>
32       <td>${o.idProduit}</td>
33       <td>${o.nom}</td>
34       <td>${o.description}</td>
35       <td>${o.prix}</td>
36       <td>
37         <a href="deleteProduit.aspx?id=${o.idProduit}">
38           supprimer </a> |
39         <a href="editProduit.aspx?id=${o.idProduit}">
40           Modifier </a>
41       </td>
42     </tr>
43   </c:forEach>
44 </table>
45 </body>
46 </html>

```

7 Conclusion

L'intégration de la solution Spring MVC a permis d'optimiser l'organisation globale du développement web. L'application est maintenant scindée de façon claire, garantissant une indépendance forte entre les fonctionnalités métier, l'accès aux bases de données, et l'affichage des informations pour l'utilisateur. Cette structure rend le projet nettement plus facile à entretenir et à étendre.